

How should a model and its tools share the work?

Our long-term aim is to solve unfamiliar tasks by breaking them into smaller steps. A recursive language model (RLM) uses code and can ask other models for help.

Before adding helpers, we test two basic questions.

What examples teach a model to find the information it needs?
Which details should code handle instead of the model?

Our test: a crafting game. To make a pickaxe, the model must look up recipes, make the parts, and assemble the item.

The game checks whether the requested item was actually made, not whether the answer sounds convincing.

Pickaxe example simplified. We use TextCraft and one small model with tools. These experiments do not yet test recursive helper calls.

Can examples teach a model how to choose its steps?

We train copies of the same model to imitate the next action in a worked example. This is **supervised fine-tuning (SFT)**.

One simplified training example

Situation: “Make a pickaxe. Its recipe is not known yet.”

Action to learn: “Ask the game for the pickaxe recipe.”

The teacher knows recipes.

It uses known recipes to plan the steps, then demonstrates lookups and actions.

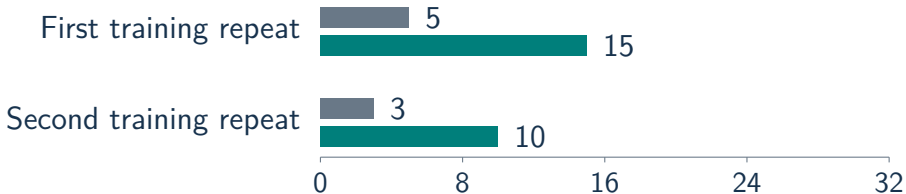
The teacher discovers recipes.

It looks up the goal's recipe, then its parts' recipes, to find the steps.

Both teachers are programs; both demonstrate lookups on the same 32 training tasks. They differ in how the route is found, action order, and text.

The discovery examples led to more finished tasks.

Each model made **32 attempts**: two on each of 16 evaluation tasks. We repeated training twice. Higher bars mean more successes.



■ Teacher knows recipes ■ Teacher discovers recipes

With changed recipes, discovery also led: **6 vs. 4** and **8 vs. 1** successes out of 16. The smaller lead is uncertain.

The first teacher's quantity error is corrected. These tests helped us develop the approach; a separate final test is still needed.

Code can handle details without choosing the plan.

A model can read the right recipe but still send the wrong ingredients. We changed the tool, with **no extra training**:

The model chooses what to make and how much.

Code fills in ingredients from a recipe already looked up.

Successful attempts (out of 16)	Model fills ingredients	Code fills ingredients
Recipe set A, first trained model	6	12
Recipe set A, second trained model	8	10
Recipe set B, first trained model	9	14
Recipe set B, second trained model	9	13

A and B change the recipes and supplies for the same eight goals. Both models learned from discovery. The smallest gain is uncertain; all outcomes are known.

The promising direction is learning and tools together.

What we learned in these small tests

Examples that discover the steps worked better than examples from a teacher that already knew them.

Letting code handle recipe details helped without changing the model's training.

What we have not shown: that this works beyond crafting, or that adding recursive helpers improves planning.

Next test: On new goals, does code help both kinds of trained model? Does the discovery advantage remain?

For discussion: What other multi-step task would make this a convincing result?

The broader comparison is queued. It is future evidence, not a result claimed here.